

Linguagens e Programação

Gramáticas

2025/2026

Instituto Superior de Engenharia do Porto

O que é uma gramática?

- Uma gramática é uma ferramenta poderosa para a descrição e análise de linguagens;
- Constituída por um conjunto de regras segundo as quais as frases válidas da linguagem são construídas;
- Constituída por vários tipos de palavras (*tokens*) normalmente reconhecidos pelo **analisador léxico**;

Definição formal

- Formalmente, uma gramática é definida pelo tuplo $G = (V, \Sigma, P, S)$, no qual:

Definição formal

- Formalmente, uma gramática é definida pelo tuplo $G = (V, \Sigma, P, S)$, no qual:
 - V – um conjunto finito, não vazio, de **variáveis** (símbolos **não terminais**);
Informalmente representado por letras maiúsculas (A, B, ...)

Definição formal

- Formalmente, uma gramática é definida pelo tuplo $G = (V, \Sigma, P, S)$, no qual:
 - V – um conjunto finito, não vazio, de **variáveis** (símbolos **não terminais**); Informalmente representado por letras maiúsculas (A, B, ...)
 - Σ – é um conjunto finito, não vazio, dito **alfabeto** ou conjunto de símbolos **terminais**; Informalmente representado por letras minúsculas (a, b, ...)

Definição formal

- Formalmente, uma gramática é definida pelo tuplo $G = (V, \Sigma, P, S)$, no qual:
 - V – um conjunto finito, não vazio, de **variáveis** (símbolos **não terminais**); Informalmente representado por letras maiúsculas (A, B, ...)
 - Σ – é um conjunto finito, não vazio, dito **alfabeto** ou conjunto de símbolos **terminais**; Informalmente representado por letras minúsculas (a, b, ...)
 - P – conjunto de **produções**, regras da gramática. A sua forma geral é a seguinte: $\alpha \rightarrow \beta$ que definem a forma como o conjunto de símbolos α podem ser substituídos pelo conjunto de símbolos β ;

Definição formal

- Formalmente, uma gramática é definida pelo tuplo $G = (V, \Sigma, P, S)$, no qual:
 - V – um conjunto finito, não vazio, de **variáveis** (símbolos **não terminais**); Informalmente representado por letras maiúsculas (A, B, ...)
 - Σ – é um conjunto finito, não vazio, dito **alfabeto** ou conjunto de símbolos **terminais**; Informalmente representado por letras minúsculas (a, b, ...)
 - P – conjunto de **produções**, regras da gramática. A sua forma geral é a seguinte: $\alpha \rightarrow \beta$ que definem a forma como o conjunto de símbolos α podem ser substituídos pelo conjunto de símbolos β ;
 - S – **símbolo inicial** a partir do qual todas as frases são derivadas.

Notação BNF

A notação BNF (*Backus Naur Form* ou *Backus Normal Form*) foi originalmente criada por **John Backus** e **Peter Naur**, no final dos anos 50, para descrever a linguagem ALGOL.

Os meta-símbolos utilizados na notação BNF são:

$::=$ — “definido como”;

$|$ — alternativa;

$< >$ — símbolos não terminais (ou variáveis)

Notação BNF

A notação BNF (*Backus Naur Form* ou *Backus Normal Form*) foi originalmente criada por **John Backus** e **Peter Naur**, no final dos anos 50, para descrever a linguagem ALGOL.

Os meta-símbolos utilizados na notação BNF são:

$::=$ — “definido como”;

$|$ — alternativa;

$< >$ — símbolos não terminais (ou variáveis)

$\langle \text{numero} \rangle ::= \langle \text{algarismo} \rangle | \langle \text{algarismo} \rangle \langle \text{numero} \rangle$

$\langle \text{algarismo} \rangle ::= 0 | 1 | 2 | \dots | 8 | 9$

Notação BNF (Exemplo avançado)

A notação BNF (*Backus Naur Form* ou *Backus Normal Form*) foi originalmente criada por **John Backus** e **Peter Naur**, no final dos anos 50, para descrever a linguagem ALGOL.

$$\begin{aligned}\langle \text{if} \rangle &::= \text{if} \langle \text{condição} \rangle \text{ then } \langle \text{instruções} \rangle \text{ endif} \\ &\quad | \text{if} \langle \text{condição} \rangle \text{ then } \langle \text{instruções} \rangle \text{ else } \langle \text{instruções} \rangle \text{ endif} \\ \langle \text{identificador} \rangle &::= \langle \text{letra} \rangle \langle \text{alfanums} \rangle | _ \langle \text{alfanums} \rangle \\ \langle \text{alfanums} \rangle &::= \langle \text{alfanum} \rangle \langle \text{alfanums} \rangle | \epsilon \\ \langle \text{alfanum} \rangle &::= \langle \text{letra} \rangle | \langle \text{algarismo} \rangle | _ \\ \langle \text{letra} \rangle &::= a | A | b | \dots | z | Z \\ \langle \text{algarismo} \rangle &::= 0 | 1 | 2 | \dots | 8 | 9\end{aligned}$$

Notação EBNF

A notação EBNF estende a notação BNF com os seguintes meta-símbolos:

- [] — parte opcional;
- { } — parte que se pode repetir 0 ou mais vezes;
- () — precedências dentro da regra;
- " " — carácter a tratar como símbolo terminal e.g., "<".

Notação EBNF

A notação EBNF estende a notação BNF com os seguintes meta-símbolos:

- [] — parte opcional;
- { } — parte que se pode repetir 0 ou mais vezes;
- () — precedências dentro da regra;
- " " — carácter a tratar como símbolo terminal e.g., "<".

<identificador> ::= (<letra> | _){(<letra> | <algarismo> | _)}

<letra> ::= a | A | b | ... | z | Z

<algarismo> ::= 0 | 1 | 2 | ... | 8 | 9

Notação EBNF (Exemplo avançado)

A notação EBNF estende a notação BNF com os seguintes meta-símbolos:

- [] — parte opcional;
- { } — parte que se pode repetir 0 ou mais vezes;
- () — precedências dentro da regra;
- " " — carácter a tratar como símbolo terminal e.g., "<".

$\langle \text{if} \rangle ::= \text{if } \langle \text{condição} \rangle \text{ then } \langle \text{instruções} \rangle [\text{else } \langle \text{instruções} \rangle] \text{ endif}$

$\langle \text{identificador} \rangle ::= (\langle \text{letra} \rangle | _)\{(\langle \text{letra} \rangle | \langle \text{algarismo} \rangle | _)\}$

$\langle \text{letra} \rangle ::= a | A | b | \dots | z | Z$

$\langle \text{algarismo} \rangle ::= 0 | 1 | 2 | \dots | 8 | 9$

Exemplo de uma gramática

<frase> → **<sintagma-nominal><sintagma-verbal>**

<sintagma-nominal> → **<artigo><nome>** | **<nome>**

<sintagma-verbal> → **<verbo><sintagma-nominal>**

<artigo> → *o* | *a* | *os* | *as*

<nome> → *Pedro* | *Maria* | *crianças* | *rapazes* | *cartas* | *futebol*

<verbo> → *conhece* | *conhecem* | *é* | *são* | *joga* | *jogam*

Exemplo de uma gramática

<frase> → <sintagma-nominal><sintagma-verbal>

<sintagma-nominal> → <artigo><nome> | <nome>

<sintagma-verbal> → <verbo><sintagma-nominal>

<artigo> → o | a | os | as

<nome> → Pedro | Maria | crianças | rapazes | cartas | futebol

<verbo> → conhece | conhecem | é | são | joga | jogam

Com estas regras (também designadas por produções) é possível analisar frases como as seguintes:

os rapazes jogam futebol

o Pedro conhece a Maria

a Maria é criança

Exemplo de uma gramática

$\langle \text{frase} \rangle \rightarrow \langle \text{sintagma-nominal} \rangle \langle \text{sintagma-verbal} \rangle$

$\langle \text{sintagma-nominal} \rangle \rightarrow \langle \text{artigo} \rangle \langle \text{nome} \rangle \mid \langle \text{nome} \rangle$

$\langle \text{sintagma-verbal} \rangle \rightarrow \langle \text{verbo} \rangle \langle \text{sintagma-nominal} \rangle$

$\langle \text{artigo} \rangle \rightarrow o \mid a \mid os \mid as$

$\langle \text{nome} \rangle \rightarrow Pedro \mid Maria \mid crianças \mid rapazes \mid cartas \mid futebol$

$\langle \text{verbo} \rangle \rightarrow conhece \mid conhecem \mid é \mid são \mid joga \mid jogam$

Derivação de frases

$u \Rightarrow v$ — diz-se que u deriva em v num passo

$u \Rightarrow^* v$ — diz-se que u deriva em v em zero ou mais passos

$u \Rightarrow^+ v$ — diz-se que u deriva em v em um ou mais passos

Exemplo de uma gramática

$\langle \text{frase} \rangle \rightarrow \langle \text{sintagma-nominal} \rangle \langle \text{sintagma-verbal} \rangle$

$\langle \text{sintagma-nominal} \rangle \rightarrow \langle \text{artigo} \rangle \langle \text{nome} \rangle \mid \langle \text{nome} \rangle$

$\langle \text{sintagma-verbal} \rangle \rightarrow \langle \text{verbo} \rangle \langle \text{sintagma-nominal} \rangle$

$\langle \text{artigo} \rangle \rightarrow o \mid a \mid os \mid as$

$\langle \text{nome} \rangle \rightarrow \textit{Pedro} \mid \textit{Maria} \mid \textit{crianças} \mid \textit{rapazes} \mid \textit{cartas} \mid \textit{futebol}$

$\langle \text{verbo} \rangle \rightarrow \textit{conhece} \mid \textit{conhecem} \mid \textit{é} \mid \textit{são} \mid \textit{joga} \mid \textit{jogam}$

A linguagem gerada por uma gramática $L(G)$ é dada pelo conjunto de todas as derivações que, partindo do estado inicial S , originam uma sequência de símbolos do alfabeto:

$$L(G) = \{u \in \Sigma^* : S \Rightarrow^* u\}$$

Derivação de frases

$\langle \text{frase} \rangle \rightarrow \langle \text{sintagma-nominal} \rangle \langle \text{sintagma-verbal} \rangle$

$\langle \text{sintagma-nominal} \rangle \rightarrow \langle \text{artigo} \rangle \langle \text{nome} \rangle \mid \langle \text{nome} \rangle$

$\langle \text{sintagma-verbal} \rangle \rightarrow \langle \text{verbo} \rangle \langle \text{sintagma-nominal} \rangle$

$\langle \text{artigo} \rangle \rightarrow o \mid a \mid os \mid as$

$\langle \text{nome} \rangle \rightarrow \textit{Pedro} \mid \textit{Maria} \mid \textit{crianças} \mid \textit{rapazes} \mid \textit{cartas} \mid \textit{futebol}$

$\langle \text{verbo} \rangle \rightarrow \textit{conhece} \mid \textit{conhecem} \mid \textit{é} \mid \textit{são} \mid \textit{joga} \mid \textit{jogam}$

$\langle \text{frase} \rangle \Rightarrow \langle \text{sintagma-nominal} \rangle \langle \text{sintagma-verbal} \rangle$

$\Rightarrow \langle \text{artigo} \rangle \langle \text{nome} \rangle \langle \text{sintagma-verbal} \rangle$

$\Rightarrow os \langle \text{nome} \rangle \langle \text{sintagma-verbal} \rangle$

$\Rightarrow os \textit{rapazes} \langle \text{sintagma-verbal} \rangle$

$\Rightarrow os \textit{rapazes} \langle \text{verbo} \rangle \langle \text{sintagma-nominal} \rangle$

Derivação de frases

$\langle \text{frase} \rangle \rightarrow \langle \text{sintagma-nominal} \rangle \langle \text{sintagma-verbal} \rangle$

$\langle \text{sintagma-nominal} \rangle \rightarrow \langle \text{artigo} \rangle \langle \text{nome} \rangle \mid \langle \text{nome} \rangle$

$\langle \text{sintagma-verbal} \rangle \rightarrow \langle \text{verbo} \rangle \langle \text{sintagma-nominal} \rangle$

$\langle \text{artigo} \rangle \rightarrow o \mid a \mid os \mid as$

$\langle \text{nome} \rangle \rightarrow Pedro \mid Maria \mid crianças \mid rapazes \mid cartas \mid futebol$

$\langle \text{verbo} \rangle \rightarrow conhece \mid conhecem \mid é \mid são \mid joga \mid jogam$

$\langle \text{frase} \rangle \Rightarrow^* os \ rapazes \langle \text{verbo} \rangle \langle \text{sintagma-nominal} \rangle$

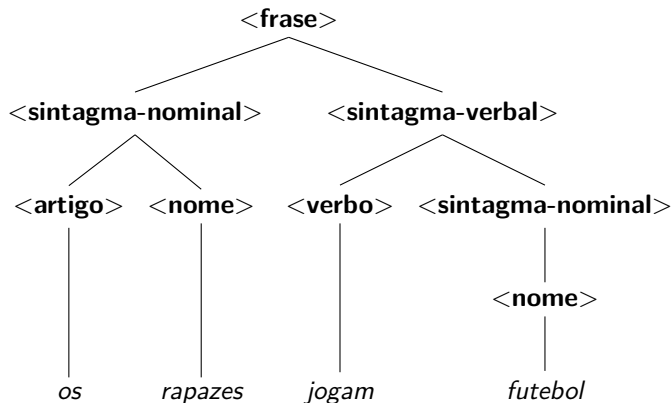
$\Rightarrow os \ rapazes \ jogam \langle \text{sintagma-nominal} \rangle$

$\Rightarrow os \ rapazes \ jogam \langle \text{nome} \rangle$

$\Rightarrow os \ rapazes \ jogam \ futebol$

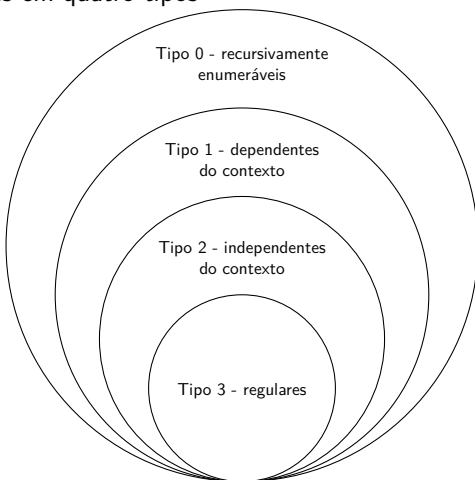
Árvores de derivação

O processo de derivação pode ser representado graficamente através de uma árvore de derivação



Hierarquia de Chomsky

Noam Chomsky (linguista americano) criou uma hierarquia para as gramáticas formais, dividindo-as em quatro tipos



Gramáticas do tipo 3

Gramáticas regulares. A produções são da forma:

$$A \rightarrow a$$

$$A \rightarrow aB$$

$$A \rightarrow \varepsilon$$

em que **A** e **B** são dois quaisquer **não terminais** singulares e **a** é um qualquer **terminal** singular.

Estas são as formas de gramáticas mais restritas em termos de poder de representação.

Nota: A produção $A \rightarrow a$ pode ser omitida, pois pode ser derivada a partir das outras duas.

Gramáticas do tipo 3

Gramáticas regulares. A produções são da forma:

$$A \rightarrow a$$

$$A \rightarrow aB$$

$$A \rightarrow \varepsilon$$

em que **A** e **B** são dois quaisquer **não terminais** singulares e **a** é um qualquer **terminal** singular.

Estas são as formas de gramáticas mais restritas em termos de poder de representação.

Lineares à direita

$$A \rightarrow a$$

$$A \rightarrow aB$$

$$A \rightarrow \varepsilon$$

Lineares à esquerda

$$A \rightarrow a$$

$$A \rightarrow Ba$$

$$A \rightarrow \varepsilon$$

Gramáticas do tipo 2

Gramáticas independentes do contexto. As produções são da forma:

$$A \rightarrow \alpha$$

em que α é uma sequência arbitrária de símbolos terminais e não terminais, sendo **A** um qualquer não terminal singular.

Tal significa que qualquer ocorrência de **A** pode ser substituída por α independentemente do contexto.

Gramáticas do tipo 2

Gramáticas independentes do contexto. A produções são da forma:

$$A \rightarrow \alpha$$

em que α é uma sequência arbitrária de símbolos terminais e não terminais, sendo **A** um qualquer não terminal singular.

Tal significa que qualquer ocorrência de **A** pode ser substituída por α independentemente do contexto.

Reconhece frases do tipo
 $\{a^n b^p d^q c^n : n, p, q \geq 0\}$

Exemplo

$$A \rightarrow aAc$$

$$A \rightarrow BD$$

$$B \rightarrow bB|\varepsilon$$

$$D \rightarrow dD|\varepsilon$$

Gramáticas do tipo 1

Gramáticas dependentes do contexto. As produções são da forma:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

em que α, β e γ são sequências arbitrárias de símbolos terminais e não terminais, sendo que γ não é nulo e A é um qualquer não terminal singular.

Estas gramáticas têm que respeitar a condição $|\alpha A \beta| \leq |\alpha \gamma \beta|$

A única exceção à regra anterior, é a produção inicial ser $S \rightarrow \varepsilon$, mas **se e só se** o S não existir do lado direito (para permitir a palavra vazia).

Gramáticas do tipo 1

Gramáticas dependentes do contexto. A produções são da forma:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

em que α, β e γ são sequências arbitrárias de símbolos terminais e não terminais, sendo que γ não é nulo e **A** é um qualquer não terminal singular.

Reconhece frases do tipo

$$\{a^n b^n c^n : n \geq 1\}$$

Exemplo

$$S \rightarrow aSBC|abC$$

$$CB \rightarrow XB$$

$$XB \rightarrow XC$$

$$XC \rightarrow BC$$

$$bB \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

Gramáticas do tipo 0

Gramáticas livres ou sem restrições. As produções são da forma:

$$\alpha \rightarrow \beta$$

em que tanto α como β são sequências arbitrárias de símbolos terminais e não terminais.

O lado esquerdo da produção não pode ser vazio.

Gramáticas do tipo 0

Gramáticas livres ou sem restrições. As produções são da forma:

$$\alpha \rightarrow \beta$$

em que tanto α como β são sequências arbitrárias de símbolos terminais e não terminais.

O lado esquerdo da produção não pode ser vazio.

Reconhece frases do tipo
 $\{a^n b^n c^n : n \geq 1\}$

Exemplo

$$S \rightarrow aSBC|abC$$

$$CB \rightarrow BC$$

$$bB \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

Exercícios

Escreva uma gramática para as seguintes linguagens:

- Números binários pares
- Números binários com o máximo de 2 zeros
- Números binários começados e terminados por 1
- Números binários em que 111 é factor do número
- Números binários capicuas

Diga que tipo de gramática escreveu e se será possível convertê-la numa gramática mais restrita (tipo maior)

Gramática ambígua

Uma gramática diz-se ambígua, se é possível obter uma mesma frase a partir de duas árvores de derivação distintas.

$$S \rightarrow aS \mid Sa \mid b$$

Gramática ambígua

Uma gramática diz-se ambígua, se é possível obter uma mesma frase a partir de duas árvores de derivação distintas.

$$S \rightarrow aS \mid Sa \mid b$$

A frase ***aba*** pode ser obtida através de qualquer uma das seguintes sequências de derivação:

$$S \Rightarrow aS \Rightarrow aSa \Rightarrow aba$$

$$S \Rightarrow Sa \Rightarrow aSa \Rightarrow aba$$

Gramática ambígua

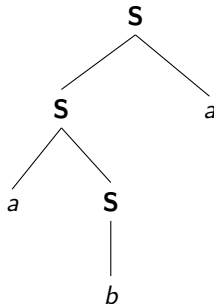
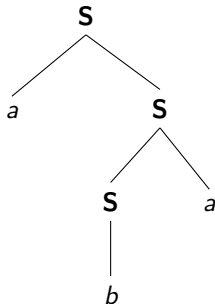
Uma gramática diz-se ambígua, se é possível obter uma mesma frase a partir de duas árvores de derivação distintas.

$$S \rightarrow aS \mid Sa \mid b$$

A frase ***aba*** pode ser obtida através de qualquer uma das seguintes sequências de derivação:

$$S \Rightarrow aS \Rightarrow aSa \Rightarrow aba$$

$$S \Rightarrow Sa \Rightarrow aSa \Rightarrow aba$$



Linguagem ambigua

- Quando a ambiguidade depende da gramática pode ser eliminada.
- Nos casos em que depende da linguagem, a ambiguidade não pode ser eliminada.

Linguagem ambigua

- Quando a ambiguidade depende da gramática pode ser eliminada.
- Nos casos em que depende da linguagem, a ambiguidade não pode ser eliminada.

Exemplo de linguagem inerentemente ambígua:

$$L = \{a^i b^j c^j : i, j \geq 1\} \cup \{a^i b^j c^j : i, j \geq 1\}$$

Linguagem ambígua

- Quando a ambiguidade depende da gramática pode ser eliminada.
- Nos casos em que depende da linguagem, a ambiguidade não pode ser eliminada.

Exemplo de linguagem inerentemente ambígua:

$$L = \{a^i b^j c^j : i, j \geq 1\} \cup \{a^i b^j c^j : i, j \geq 1\}$$

Esta linguagem pode ser reconhecida pela seguinte gramática:

S $\rightarrow$ **AB** | **CD**

A $\rightarrow$ **aAb** | **ab**

B $\rightarrow$ **cB** | **c**

C $\rightarrow$ **aC** | **a**

D $\rightarrow$ **bDc** | **bc**

Esta linguagem é ambígua para palavras em que $i = j$

Linguagem ambigua

$$S \rightarrow AB \mid CD$$

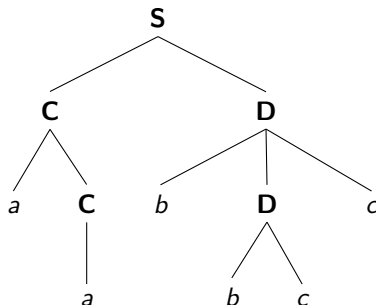
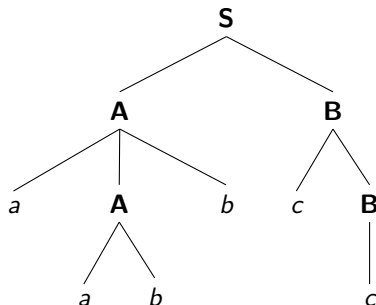
$$A \rightarrow aAb \mid ab$$

$$B \rightarrow cB \mid c$$

$$C \rightarrow aC \mid a$$

$$D \rightarrow bDc \mid bc$$

$$S \Rightarrow AB \Rightarrow aAbB \Rightarrow aabbB \Rightarrow aabbcB \Rightarrow aabbcc$$

$$S \Rightarrow CD \Rightarrow aCD \Rightarrow aaD \Rightarrow aabDc \Rightarrow aabbcc$$


Eliminação de Ambiguidade — Dupla Recursividade

$$E \rightarrow \langle E \rangle \langle OP \rangle \langle E \rangle \mid \langle ID \rangle$$
$$OP \rightarrow + \mid - \mid \times \mid \div$$
$$ID \rightarrow x \mid y \mid z$$

Eliminação de Ambiguidade — Dupla Recursividade

$$E \rightarrow \langle E \rangle \langle OP \rangle \langle E \rangle \mid \langle ID \rangle$$

$$OP \rightarrow + \mid - \mid \times \mid \div$$

$$ID \rightarrow x \mid y \mid z$$

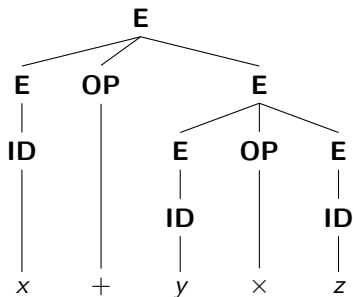
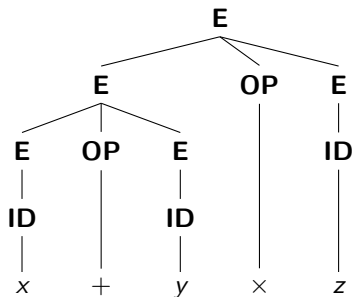
$$\begin{aligned} \langle E \rangle &\Rightarrow \langle E \rangle \langle OP \rangle \langle E \rangle \Rightarrow \langle E \rangle \langle OP \rangle \langle E \rangle \langle OP \rangle \langle E \rangle \Rightarrow \\ &\Rightarrow \langle ID \rangle \langle OP \rangle \langle E \rangle \langle OP \rangle \langle E \rangle \Rightarrow x \langle OP \rangle \langle E \rangle \langle OP \rangle \langle E \rangle \Rightarrow \\ &\Rightarrow x + \langle E \rangle \langle OP \rangle \langle E \rangle \Rightarrow x + \langle ID \rangle \langle OP \rangle \langle E \rangle \Rightarrow x + y \langle OP \rangle \langle E \rangle \Rightarrow \\ &\Rightarrow x + y \times \langle E \rangle \Rightarrow x + y \times \langle ID \rangle \Rightarrow x + y \times z \end{aligned}$$

$$\begin{aligned} \langle E \rangle &\Rightarrow \langle E \rangle \langle OP \rangle \langle E \rangle \Rightarrow \langle ID \rangle \langle OP \rangle \langle E \rangle \Rightarrow x \langle OP \rangle \langle E \rangle \Rightarrow x + \langle E \rangle \Rightarrow \\ &\Rightarrow x + \langle E \rangle \langle OP \rangle \langle E \rangle \Rightarrow x + \langle ID \rangle \langle OP \rangle \langle E \rangle \Rightarrow x + y \langle OP \rangle \langle E \rangle \Rightarrow \\ &\Rightarrow x + y \times \langle E \rangle \Rightarrow x + y \times \langle ID \rangle \Rightarrow x + y \times z \end{aligned}$$

Eliminação de Ambiguidade — Dupla Recursividade

$$E \rightarrow \langle E \rangle \langle OP \rangle \langle E \rangle \mid \langle ID \rangle$$

$$OP \rightarrow + \mid - \mid \times \mid \div$$

$$ID \rightarrow x \mid y \mid z$$


Eliminação de Ambiguidade — Dupla Recursividade

Eliminação da ambiguidade retirando uma das recursividades

$$E \rightarrow \langle E \rangle \langle OP \rangle \langle F \rangle \mid \langle F \rangle$$

$$F \rightarrow \langle ID \rangle$$

$$OP \rightarrow + \mid - \mid \times \mid \div$$

$$ID \rightarrow x \mid y \mid z$$

Eliminação de Ambiguidade — Dupla Recursividade

Eliminação da ambiguidade retirando uma das recursividades

$$E \rightarrow \langle E \rangle \langle OP \rangle \langle F \rangle \mid \langle F \rangle$$

$$F \rightarrow \langle ID \rangle$$

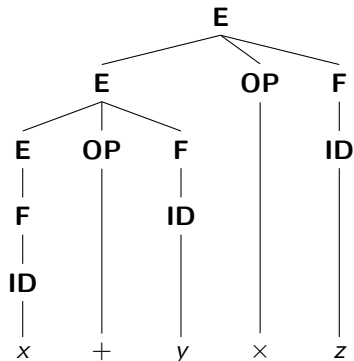
$$OP \rightarrow + \mid - \mid \times \mid \div$$

$$ID \rightarrow x \mid y \mid z$$

$$\begin{aligned} \langle E \rangle &\Rightarrow \langle E \rangle \langle OP \rangle \langle F \rangle \Rightarrow \langle E \rangle \langle OP \rangle \langle F \rangle \langle OP \rangle \langle F \rangle \\ &\Rightarrow \langle F \rangle \langle OP \rangle \langle F \rangle \langle OP \rangle \langle F \rangle \Rightarrow \langle ID \rangle \langle OP \rangle \langle F \rangle \langle OP \rangle \langle F \rangle \\ &\Rightarrow x \langle OP \rangle \langle F \rangle \langle OP \rangle \langle F \rangle \Rightarrow x + \langle F \rangle \langle OP \rangle \langle F \rangle \Rightarrow x + \langle ID \rangle \langle OP \rangle \langle F \rangle \\ &\Rightarrow x + y \langle OP \rangle \langle F \rangle \Rightarrow x + y \times \langle F \rangle \Rightarrow x + y \times z \end{aligned}$$

Eliminação de Ambiguidade — Dupla Recursividade

Eliminação da ambiguidade retirando uma das recursividades

$$E \rightarrow \langle E \rangle \langle OP \rangle \langle F \rangle \mid \langle F \rangle$$
$$F \rightarrow \langle ID \rangle$$
$$OP \rightarrow + \mid - \mid \times \mid \div$$
$$ID \rightarrow x \mid y \mid z$$


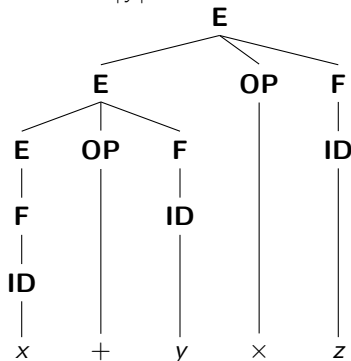
Eliminação de Ambiguidade — Dupla Recursividade

Eliminação da ambiguidade retirando uma das recursividades

$$E \rightarrow \langle E \rangle \langle OP \rangle \langle F \rangle \mid \langle F \rangle$$

$$F \rightarrow \langle ID \rangle$$

$$OP \rightarrow + \mid - \mid \times \mid \div$$

$$ID \rightarrow x \mid y \mid z$$


A precedência dos operadores está correcta?

Precedência de Operadores

$$E \rightarrow \langle E \rangle + \langle T \rangle \mid \langle E \rangle - \langle T \rangle \mid \langle T \rangle$$
$$T \rightarrow \langle T \rangle \times \langle F \rangle \mid \langle T \rangle \div \langle F \rangle \mid \langle F \rangle$$
$$F \rightarrow \langle ID \rangle \mid (\langle E \rangle)$$
$$ID \rightarrow x \mid y \mid z$$

Precedência de Operadores

$$E \rightarrow \langle E \rangle + \langle T \rangle \mid \langle E \rangle - \langle T \rangle \mid \langle T \rangle$$

$$T \rightarrow \langle T \rangle \times \langle F \rangle \mid \langle T \rangle \div \langle F \rangle \mid \langle F \rangle$$

$$F \rightarrow \langle ID \rangle \mid (\langle E \rangle)$$

$$ID \rightarrow x \mid y \mid z$$

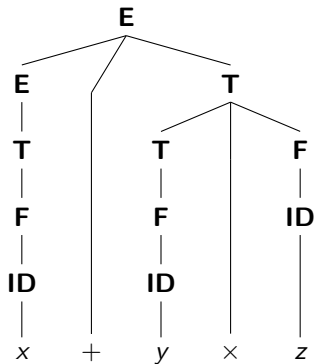
$$\langle E \rangle \Rightarrow \langle E \rangle + \langle T \rangle \Rightarrow \langle T \rangle + \langle T \rangle \Rightarrow \langle F \rangle + \langle T \rangle$$

$$\Rightarrow \langle ID \rangle + \langle T \rangle \Rightarrow x + \langle T \rangle \Rightarrow x + \langle T \rangle \times \langle F \rangle$$

$$\Rightarrow x + \langle F \rangle \times \langle F \rangle \Rightarrow x + \langle ID \rangle \times \langle F \rangle$$

$$\Rightarrow x + y \times \langle F \rangle \Rightarrow x + y \times \langle ID \rangle \Rightarrow x + y \times z$$

Precedência de Operadores

$$E \rightarrow \langle E \rangle + \langle T \rangle \mid \langle E \rangle - \langle T \rangle \mid \langle T \rangle$$
$$T \rightarrow \langle T \rangle \times \langle F \rangle \mid \langle T \rangle \div \langle F \rangle \mid \langle F \rangle$$
$$F \rightarrow \langle ID \rangle \mid (\langle E \rangle)$$
$$ID \rightarrow x \mid y \mid z$$


Ambiguidade do If

$$S \rightarrow \text{if}\langle C \rangle \text{ then}\langle S \rangle$$
$$S \rightarrow \text{if}\langle C \rangle \text{ then}\langle S \rangle \text{ else}\langle S \rangle$$
$$S \rightarrow x$$
$$C \rightarrow 0 \mid 1$$

Ambiguidade do If

$$S \rightarrow \text{if}\langle C \rangle \text{ then}\langle S \rangle$$
$$S \rightarrow \text{if}\langle C \rangle \text{ then}\langle S \rangle \text{ else}\langle S \rangle$$
$$S \rightarrow x$$
$$C \rightarrow 0 \mid 1$$

if 0 then if 1 then x else x"

Ambiguidade do If

$$S \rightarrow \text{if}\langle C \rangle \text{ then}\langle S \rangle$$

$$S \rightarrow \text{if}\langle C \rangle \text{ then}\langle S \rangle \text{ else}\langle S \rangle$$

$$S \rightarrow x$$

$$C \rightarrow 0 \mid 1$$

if 0 then if 1 then x else x"

$$\langle S \rangle \Rightarrow \text{if}\langle C \rangle \text{ then}\langle S \rangle \Rightarrow \text{if } 0 \text{ then}\langle S \rangle$$

$$\Rightarrow \text{if } 0 \text{ then if}\langle C \rangle \text{ then}\langle S \rangle \text{ else}\langle S \rangle \Rightarrow \text{if } 0 \text{ then if } 1 \text{ then}\langle S \rangle \text{ else}\langle S \rangle$$

$$\Rightarrow \text{if } 0 \text{ then if } 1 \text{ then } x \text{ else}\langle S \rangle \Rightarrow \text{if } 0 \text{ then if } 1 \text{ then } x \text{ else } x$$

$$\langle S \rangle \Rightarrow \text{if}\langle C \rangle \text{ then}\langle S \rangle \text{ else}\langle S \rangle \Rightarrow \text{if } 0 \text{ then}\langle S \rangle \text{ else}\langle S \rangle$$

$$\Rightarrow \text{if } 0 \text{ then if}\langle C \rangle \text{ then}\langle S \rangle \text{ else}\langle S \rangle \Rightarrow \text{if } 0 \text{ then if } 1 \text{ then}\langle S \rangle \text{ else}\langle S \rangle$$

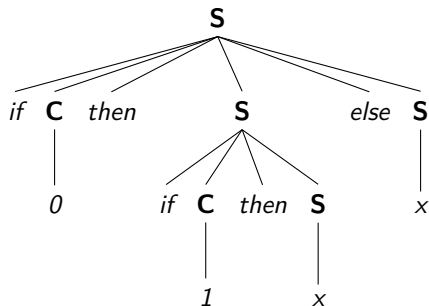
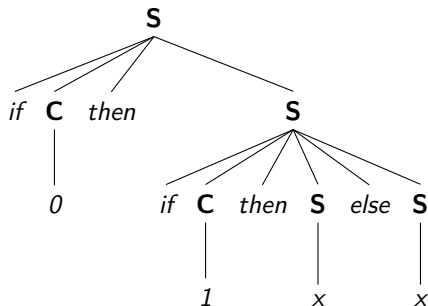
$$\Rightarrow \text{if } 0 \text{ then if } 1 \text{ then } x \text{ else}\langle S \rangle \Rightarrow \text{if } 0 \text{ then if } 1 \text{ then } x \text{ else } x$$

Ambiguidade do If

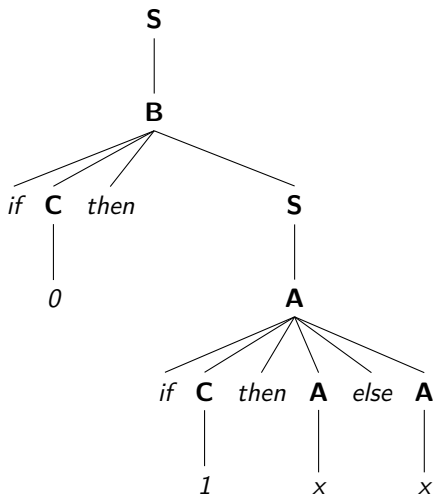
$$S \rightarrow \text{if} \langle C \rangle \text{ then} \langle S \rangle$$

$$S \rightarrow \text{if} \langle C \rangle \text{ then} \langle S \rangle \text{ else} \langle S \rangle$$

$$S \rightarrow x$$

$$C \rightarrow 0 \mid 1$$


Ambiguidade do If — Resolução



$$S \rightarrow \langle A \rangle \mid \langle B \rangle$$

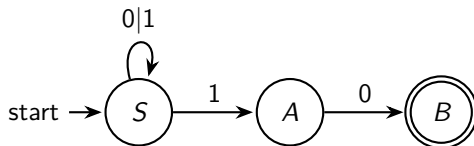
$$A \rightarrow \text{if} \langle C \rangle \text{ then} \langle A \rangle \text{ else} \langle A \rangle \mid x$$

$$B \rightarrow \text{if} \langle C \rangle \text{ then} \langle S \rangle \mid$$

$$\text{if} \langle C \rangle \text{ then} \langle A \rangle \text{ else} \langle B \rangle$$

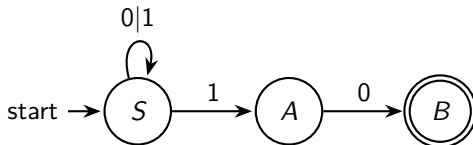
$$C \rightarrow 0 \mid 1$$

Conversão de Autômatos Finitos em Gramáticas



- Criar uma produção (não terminal) para cada estado;
- Para cada transição, criar uma alternativa na produção;
- Todos os estados finais podem receber a cadeia vazia.

Conversão de Autômatos Finitos em Gramáticas



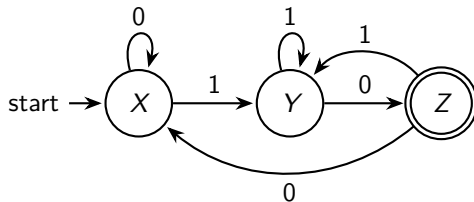
- Criar uma produção (não terminal) para cada estado;
- Para cada transição, criar uma alternativa na produção;
- Todos os estados finais podem receber a cadeia vazia.

$$S \rightarrow 0S \mid 1S \mid 1A$$

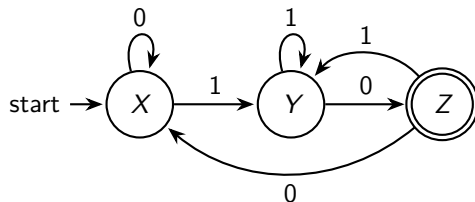
$$A \rightarrow 0B$$

$$B \rightarrow \varepsilon$$

Conversão de Autômatos Finitos em Gramáticas



Conversão de Autômatos Finitos em Gramáticas



$$X \rightarrow 0X \mid 1Y$$

$$Y \rightarrow 0Z \mid 1Y$$

$$Z \rightarrow 0X \mid 1Y \mid \varepsilon$$

Conversão de Gramáticas em Autômatos Finitos

- A gramática tem que ser do tipo 3 e ser linear à direita;
- Criar um estado para cada não terminal;
- Criar transições para as produções tal que:
 - As produções do tipo $X \rightarrow aY$ dão transições do tipo $\delta(X, a) = Y$;
 - As produções do tipo $X \rightarrow \varepsilon$ implicam que X seja um estado final;
 - As produções do tipo $X \rightarrow a$ implicam a transição para um novo estado final sem transições.

Conversão de Gramáticas em Autômatos Finitos

- A gramática tem que ser do tipo 3 e ser linear à direita;
- Criar um estado para cada não terminal;
- Criar transições para as produções tal que:
 - As produções do tipo $X \rightarrow aY$ dão transições do tipo $\delta(X, a) = Y$;
 - As produções do tipo $X \rightarrow \varepsilon$ implicam que X seja um estado final;
 - As produções do tipo $X \rightarrow a$ implicam a transição para um novo estado final sem transições.

$$A \rightarrow 1A \mid 0B$$

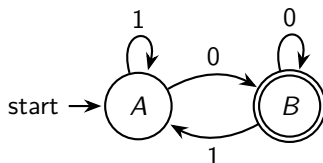
$$B \rightarrow 0B \mid 1A \mid \varepsilon$$

Conversão de Gramáticas em Autômatos Finitos

- A gramática tem que ser do tipo 3 e ser linear à direita;
- Criar um estado para cada não terminal;
- Criar transições para as produções tal que:
 - As produções do tipo $X \rightarrow aY$ dão transições do tipo $\delta(X, a) = Y$;
 - As produções do tipo $X \rightarrow \varepsilon$ implicam que X seja um estado final;
 - As produções do tipo $X \rightarrow a$ implicam a transição para um novo estado final sem transições.

$$A \rightarrow 1A \mid 0B$$

$$B \rightarrow 0B \mid 1A \mid \varepsilon$$



Parsers em descida recursivo

- A descida recursiva pode ser usada, para implementar *parsers* preditivos;
- Um *parser* preditivo é capaz de escolher a produção a aplicar simplesmente sabendo o **não terminal** actual e o **terminal** a ser processado;

Parsers em descida recursivo

- A descida recursiva pode ser usada, para implementar *parsers* preditivos;
- Um *parser* preditivo é capaz de escolher a produção a aplicar simplesmente sabendo o **não terminal** actual e o **terminal** a ser processado;
- Este tipo de gramáticas são chamadas de LL(1), onde:
 - o primeiro “L” indica que a frase é processada da esquerda para a direita;
 - o segundo “L” indica que se usa a derivação mais à esquerda;
 - o (1) indica o número de terminais de avanço necessários para escolher entre produções alternativas;
- Todas as gramáticas do tipo 3 podem ser convertidas em LL(1).

Parsers em descida recursivo

Para construir um *parser* em descida recursiva é necessário

- criar uma função para processar cada um dos não terminais;
- para cada não terminal, processar os *starters* e chamar as funções relativas aos não terminais que se lhes seguem;

$$\mathbf{S} \rightarrow a\mathbf{A} \mid c\mathbf{S} \mid d\mathbf{S}$$

$$\mathbf{A} \rightarrow a\mathbf{A} \mid b\mathbf{B} \mid c\mathbf{S} \mid d\mathbf{S}$$

$$\mathbf{B} \rightarrow a\mathbf{B} \mid b\mathbf{B} \mid c\mathbf{B} \mid d\mathbf{B} \mid \varepsilon$$

Parsers em descida recursivo

```
%{
    enum{TOKEN_A,TOKEN_B,TOKEN_C,TOKEN_D,OUTRO,FIM};

    int token, nerros=0;

    void s(); /* protótipos */
    void a();
    void b();
}%}

%%

a|A      return TOKEN_A;
b|B      return TOKEN_B;
c|C      return TOKEN_C;
d|D      return TOKEN_D;
.        return OUTRO;
\n       return FIM;
<<EOF>>  return FIM;

%%
```

Parsers em descida recursivo

```
void erro(char *s)
{
    nerros++;
    printf("%s<-Erro %d: %s\n",yytext,nerros,s);
}

void getToken()
{
    printf("%s",yytext);
    token=yylex();
}

int main()
{
    getToken(); /* vai buscar o 1º token */
    s();         /* chama a produção inicial */

    if (nerros==0 && token==FIM)
        printf("\nExpressão válida \n");
    else
        erro("símbolo não reconhecido (esperava a,b,c,d ou fim)");
    return 0;
}
```

Parsers em descida recursivo

```
void s() /* S-> aA | cS | dS */
{
    switch (token) {
        case TOKEN_A : getToken(); a(); break;
        case TOKEN_C :
        case TOKEN_D : getToken(); s(); break;
        default      :
            erro("símbolo não reconhecido (esperava a,c ou d)");
    }
}

void a() /* A-> aA | bB | cS | dS */
{
    switch (token) {
        case TOKEN_A : getToken(); a(); break;
        case TOKEN_B : getToken(); b(); break;
        case TOKEN_C :
        case TOKEN_D : getToken(); s(); break;
        default      :
            erro("símbolo não reconhecido (esperava a,b,c ou d)");
    }
}
```

Parsers em descida recursivo

```
void b() /* B-> aB | bB | cB | dB | ε */
{
    switch (token) {
        case TOKEN_A :
        case TOKEN_B :
        case TOKEN_C :
        case TOKEN_D : getToken(); b(); break;
    }
    /* como pode ser vazio não dá erro */
}
```